

Experiment 2.1 – Redux Content Manager

Step 1

Open **Visual Studio Code**.

Step 2

Open **Terminal**.

```
Terminal  
↓  
New Terminal
```

Step 3

Check Node.js installation.

```
node -v  
npm -v
```

Step 4

Create a new React project.

```
npx create-react-app redux-content-manager
```

If prompted, type:

```
y
```

Step 5

Move inside the project.

```
cd redux-content-manager
```

Step 6

Open the project.

```
code .
```

Step 7

Start the React application.

```
npm start
```

Step 8

Stop the server.

```
Ctrl + C
```

Step 9

Install Redux Toolkit.

```
npm install @reduxjs/toolkit react-redux
```

Step 10

Start the application again.

```
npm start
```

Step 11

Inside the **src** folder, create the following folders.

```
src
```

```
components
```

```
store
```

Step 12

Inside the **store** folder, create a file named:

```
store.js
```

Paste the following code.

```
import { configureStore } from "@reduxjs/toolkit";
import postsReducer from "../postsSlice";

export const store = configureStore({
  reducer: {
    posts: postsReducer,
  },
});
```

Save the file.

Step 13

Inside the **store** folder, create another file named:

postsSlice.js

Paste the following code.

```
import { createSlice } from "@reduxjs/toolkit";

const initialState = {
  posts: [],
};

const postsSlice = createSlice({
  name: "posts",
  initialState,

  reducers: {
    addPost: (state, action) => {
      state.posts.push(action.payload);
    },

    updatePost: (state, action) => {
      const index = state.posts.findIndex(
        (post) => post.id === action.payload.id
      );

      if (index !== -1) {
        state.posts[index] = action.payload;
      }
    },

    deletePost: (state, action) => {
      state.posts = state.posts.filter(
        (post) => post.id !== action.payload
      );
    },
  },
});

export const {
  addPost,
  updatePost,
  deletePost,
} = postsSlice.actions;

export default postsSlice.reducer;
```

Save the file.

Step 14

Open

src

↓
index.js

Replace the existing code with:

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "../App";

import { Provider } from "react-redux";
import { store } from "../store/store";

const root = ReactDOM.createRoot(
  document.getElementById("root")
);

root.render(
  <Provider store={store}>
    <App />
  </Provider>
);
```

Save the file.

Step 15

Inside the **components** folder, create a new file.

PostList.js

Paste the following code.

```
import { useState } from "react";
import { useSelector, useDispatch } from "react-redux";
import {
  addPost,
  updatePost,
  deletePost,
} from "../../store/postsSlice";

function PostList() {
  const posts = useSelector((state) => state.posts.posts);
  const dispatch = useDispatch();

  const [text, setText] = useState("");
  const [editId, setEditId] = useState(null);

  const handleSubmit = () => {
    if (text.trim() === "") return;

    if (editId) {
      dispatch(
        updatePost({
          id: editId,
          title: text,
        })
      );
      setEditId(null);
    } else {
      dispatch(
        addPost({
```

```

        id: Date.now(),
        title: text,
      })
    );
  }

  setText("");
};

const handleEdit = (post) => {
  setText(post.title);
  setEditId(post.id);
};

return (
  <div className="container">
    <input
      type="text"
      placeholder="Enter Post Title"
      value={text}
      onChange={ (e) => setText(e.target.value) }
    />

    <button onClick={handleSubmit}>
      {editId ? "Update Post" : "Add Post"}
    </button>

    {posts.map((post) => (
      <div key={post.id} className="card">
        <h3>{post.title}</h3>

        <button onClick={() => handleEdit(post)}>
          Edit
        </button>

        <button
          onClick={() =>
            dispatch(deletePost(post.id))
          }
        >
          Delete
        </button>
      </div>
    ))}
  </div>
);
}

export default PostList;

```

Save the file.

Step 16

Open

```
src
↓
App.js
```

Replace the existing code with:

```
import "../App.css";
import PostList from "../components/PostList";

function App() {
  return (
    <div className="App">
      <h1>Redux Content Manager</h1>
      <PostList />
    </div>
  );
}

export default App;
```

Save the file.

Step 17

Open

```
src
↓
App.css
```

Replace the existing code with:

```
body {
  margin: 0;
  font-family: Arial, sans-serif;
  background: #eef2ff;
}

.App {
  text-align: center;
  padding: 30px;
}

.container {
  width: 500px;
  margin: auto;
}

input {
  width: 70%;
  padding: 10px;
  margin-right: 10px;
}

button {
  padding: 10px 15px;
  margin: 5px;
  cursor: pointer;
}

.card {
  background: white;
  padding: 15px;
  margin-top: 15px;
  border-radius: 8px;
  box-shadow: 0px 2px 6px rgba(0,0,0,0.2);
}
```

Save the file.

Step 18

Open the terminal.

Run the application.

```
npm start
```

Step 19

Test the application.

- Enter a post title.
 - Click **Add Post**.
 - Verify that the post is displayed.
 - Click **Edit** to modify the post.
 - Click **Update Post** to save the changes.
 - Click **Delete** to remove the selected post.
-

Experiment 2.2 – Redux Selectors & Performance Optimization

Step 1

Open Visual Studio Code.

Step 2

Open Terminal.

```
Terminal  
↓  
New Terminal
```

Step 3

Create a new React project.

```
npx create-react-app redux-selector-dashboard
```

If prompted, type:

```
y
```

Step 4

Move inside the project.

```
cd redux-selector-dashboard
```

Step 5

Open the project.

```
code .
```

Step 6

Install Redux Toolkit and React Redux.

```
npm install @reduxjs/toolkit react-redux
```

Step 7

Start the React application.

```
npm start
```

Step 8

Inside the **src** folder, create the following folders.

```
src
components
store
selectors
```

Step 9

Inside the **store** folder, create a file.

```
store.js
```

Paste the following code.

```
import { configureStore } from "@reduxjs/toolkit";
import postsReducer from "../postsSlice";

export const store = configureStore({
  reducer: {
    posts: postsReducer,
  },
});
```

Save the file.

Step 10

Inside the **store** folder, create another file.

```
postsSlice.js
```

Paste the following code.

```
import { createSlice } from "@reduxjs/toolkit";

const initialState = {
  posts: [
    { id: 1, title: "React Basics" },
    { id: 2, title: "Redux Toolkit" },
    { id: 3, title: "JavaScript ES6" },
  ],
};
```

```
    { id: 4, title: "Machine Learning" },
    { id: 5, title: "Node.js" },
  ],
};

const postsSlice = createSlice({
  name: "posts",
  initialState,
  reducers: {},
});

export default postsSlice.reducer;
```

Save the file.

Step 11

Inside the **selectors** folder, create a file.

selectors.js

Paste the following code.

```
import { createSelector } from "@reduxjs/toolkit";

export const selectPosts = (state) => state.posts.posts;

export const totalPosts = createSelector(
  [selectPosts],
  (posts) => posts.length
);

export const shortPosts = createSelector(
  [selectPosts],
  (posts) =>
    posts.filter(
      (post) => post.title.length < 15
    ).length
);

export const longPosts = createSelector(
  [selectPosts],
  (posts) =>
    posts.filter(
      (post) => post.title.length >= 15
    ).length
);
```

Save the file.

Step 12

Open

src
↓

index.js

Replace the existing code with:

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "../App";

import { Provider } from "react-redux";
import { store } from "../store/store";

const root = ReactDOM.createRoot(
  document.getElementById("root")
);

root.render(
  <Provider store={store}>
    <App />
  </Provider>
);
```

Save the file.

Step 13

Inside the **components** folder, create a new file.

Dashboard.js

Paste the following code.

```
import { useState } from "react";
import { useSelector } from "react-redux";

import {
  selectPosts,
  totalPosts,
  shortPosts,
  longPosts,
} from "../selectors/selectors";

function Dashboard() {
  const posts = useSelector(selectPosts);

  const total = useSelector(totalPosts);
  const short = useSelector(shortPosts);
  const long = useSelector(longPosts);

  const [search, setSearch] = useState("");

  const filteredPosts = posts.filter((post) =>
    post.title.toLowerCase().includes(search.toLowerCase())
  );

  return (
    <div className="container">
      <input
        type="text"
        placeholder="Search Posts"
        value={search}
        onChange={(e) => setSearch(e.target.value)}
      />
    </div>
  );
}
```

```

    <div className="cards">

      <div className="card">
        <h3>Total Posts</h3>
        <p>{total}</p>
      </div>

      <div className="card">
        <h3>Short Posts</h3>
        <p>{short}</p>
      </div>

      <div className="card">
        <h3>Long Posts</h3>
        <p>{long}</p>
      </div>

    </div>

    <h2>Posts</h2>

    {filteredPosts.map((post) => (
      <div key={post.id} className="post">
        {post.title}
      </div>
    ))}

  </div>
);
}

export default Dashboard;

```

Save the file.

Step 14

Open

```

src
↓
App.js

```

Replace the existing code with:

```

import './App.css';
import Dashboard from './components/Dashboard';

function App() {
  return (
    <div className="App">
      <h1>Redux Selector Dashboard</h1>

      <Dashboard />
    </div>
  );
}

export default App;

```

Save the file.

Step 15

Open

```
src  
↓  
App.css
```

Replace the existing code with:

```
body{  
  margin:0;  
  font-family:Arial,sans-serif;  
  background:#eef2ff;  
}  
  
.App{  
  text-align:center;  
  padding:30px;  
}  
  
.container{  
  width:700px;  
  margin:auto;  
}  
  
input{  
  width:100%;  
  padding:12px;  
  margin-bottom:20px;  
}  
  
.cards{  
  display:flex;  
  justify-content:space-between;  
  margin-bottom:20px;  
}  
  
.card{  
  width:30%;  
  background:white;  
  padding:20px;  
  border-radius:10px;  
  box-shadow:0 2px 6px rgba(0,0,0,.2);  
}  
  
.post{  
  background:white;  
  padding:15px;  
  margin-top:10px;  
  border-radius:8px;  
  box-shadow:0 2px 5px rgba(0,0,0,.15);  
}
```

Save the file.

Step 16

Run the application.

```
npm start
```

Step 17

Verify the application.

- The dashboard should display:
 - Total Posts
 - Short Posts
 - Long Posts
 - Enter text in the search box.
 - Matching posts should be displayed.
 - Verify that the statistics remain correct.
-

Final Project Structure

```
redux-selector-dashboard
```

```
src
├── components
│   └── Dashboard.js
├── selectors
│   └── selectors.js
├── store
│   ├── store.js
│   └── postsSlice.js
├── App.js
├── App.css
├── index.js
└── index.css
```

Experiment 3.1 – JWT Authentication

Step 1

Open **Visual Studio Code**.

Step 2

Open **Terminal**.

```
Terminal  
↓  
New Terminal
```

Step 3

Create a new React project.

```
npm create-react-app jwt-authentication
```

If prompted, type:

```
y
```

Step 4

Move inside the project.

```
cd jwt-authentication
```

Step 5

Open the project.

```
code .
```

Step 6

Install React Router.

```
npm install react-router-dom
```

Step 7

Start the React application.

```
npm start
```

Step 8

Inside the **src** folder create the following folders.

```
src
```

```
components
```

```
pages
```

```
services
```

```
utils
```

Step 9

Inside the **services** folder create a file.

```
authService.js
```

Paste the following code.

```
const user = {
  username: "admin",
  password: "admin123",
  email: "admin@gmail.com",
  role: "Administrator",
};

export const authenticateUser = (username, password) => {

  if (
    username === user.username &&
    password === user.password
  ) {

    const header = btoa(
      JSON.stringify({
        alg: "HS256",
        typ: "JWT",
      })
    );

    const payload = btoa(
      JSON.stringify({
        username: user.username,
        email: user.email,
        role: user.role,
      })
    );

    const signature = btoa("jwt-secret");

    return `${header}.${payload}.${signature}`;
  }
}
```

```
    return null;
};
```

Save the file.

Step 10

Inside the **utils** folder create a file.

token.js

Paste the following code.

```
export const getUser = () => {
    const token = localStorage.getItem("token");
    if (!token) return null;
    return JSON.parse(
        atob(token.split(".")[1])
    );
};

export const logout = () => {
    localStorage.removeItem("token");
};
```

Save the file.

Step 11

Inside the **components** folder create a file.

Navbar.js

Paste the following code.

```
import { logout } from "../utils/token";

function Navbar() {
    return (
        <nav>
            <h2>JWT Authentication</h2>
            <button onClick={logout}>
                Logout
            </button>
        </nav>
    );
}
```

```
        </nav>

    );
}

export default Navbar;
```

Save the file.

Step 12

Inside the **pages** folder create a file.

Login.js

Paste the following code.

```
import { useState } from "react";
import { useNavigate } from "react-router-dom";
import { authenticateUser } from "../services/authService";

function Login() {

    const navigate = useNavigate();

    const [username, setUsername]=useState("");
    const [password, setPassword]=useState("");

    const handleLogin=()=>{

        const token=
        authenticateUser(username,password);

        if(token){

            localStorage.setItem("token",token);

            navigate("/dashboard");

        }

        else{

            alert("Invalid Credentials");

        }

    };

    return (

<div>

<h1>Login</h1>

<input

placeholder="Username"
```

```

value={username}

onChange={ (e)=>setUsername (e.target.value) }

/>

<input

type="password"

placeholder="Password"

value={password}

onChange={ (e)=>setPassword (e.target.value) }

/>

<button onClick={handleLogin}>

Login

</button>

</div>

);

}

export default Login;

```

Save the file.

Step 13

Inside the **pages** folder create a new file.

Dashboard.js

Paste the following code.

```

import Navbar from "../components/Navbar";
import { getUser } from "../utils/token";

function Dashboard() {

  const user = getUser();

  return (

    <div>

      <Navbar />

      <h1>Dashboard</h1>

      <h3>Welcome {user?.username}</h3>

      <p>Email : {user?.email}</p>

      <p>Role : {user?.role}</p>

```

```
        </div>

    );
}

export default Dashboard;
```

Save the file.

Step 14

Open

```
src
↓
App.js
```

Replace the existing code with:

```
import {
  BrowserRouter,
  Routes,
  Route,
} from "react-router-dom";

import Login from "../pages/Login";
import Dashboard from "../pages/Dashboard";

import "../App.css";

function App() {

  return (

    <BrowserRouter>

      <Routes>

        <Route
          path="/"
          element={<Login />}
        />

        <Route
          path="/dashboard"
          element={<Dashboard />}
        />

      </Routes>

    </BrowserRouter>

  );
}

export default App;
```

Save the file.

Step 15

Open

```
src  
↓  
App.css
```

Replace the existing code with:

```
body{  
margin:0;  
font-family:Arial,sans-serif;  
background:#eef2ff;  
}  
  
div{  
text-align:center;  
padding:20px;  
}  
  
input{  
width:250px;  
padding:10px;  
margin:10px;  
}  
  
button{  
padding:10px 20px;  
cursor:pointer;  
background:#4f46e5;  
color:white;  
border:none;  
border-radius:5px;  
}  
  
nav{  
display:flex;  
justify-content:space-between;  
align-items:center;  
background:#4f46e5;  
padding:15px;  
color:white;  
}  
  
h1{  
color:#4f46e5;  
}
```

Save the file.

Step 16

Open the terminal.

Run the application.

npm start

Step 17

Enter the following credentials.

Username : admin

Password : admin123

Click the **Login** button.

Step 18

Verify the output.

- Login page should be displayed.
 - Enter valid credentials.
 - JWT token should be generated.
 - Token should be stored in **Local Storage**.
 - Dashboard should display Username, Email and Role.
 - Click **Logout** to remove the token.
-

Final Project Structure

jwt-authentication

```
src
├── components
│   └── Navbar.js
├── pages
│   ├── Login.js
│   └── Dashboard.js
├── services
│   └── authService.js
├── utils
│   └── token.js
├── App.js
├── App.css
├── index.js
└── index.css
```

Experiment 3.2 – Role-Based Access Control (RBAC)

Step 1

Open **Visual Studio Code**.

Step 2

Open **Terminal**.

```
Terminal  
↓  
New Terminal
```

Step 3

Create a new React project.

```
npx create-react-app rbac-authentication
```

If prompted, type:

```
y
```

Step 4

Move inside the project.

```
cd rbac-authentication
```

Step 5

Open the project.

```
code .
```

Step 6

Install React Router.

```
npm install react-router-dom
```

Step 7

Start the application.

```
npm start
```

Step 8

Inside the **src** folder create the following folders.

```
src
```

```
components
```

```
pages
```

```
routes
```

```
services
```

```
utils
```

Step 9

Inside the **services** folder create a file.

```
authService.js
```

Paste the following code.

```
const users = [
  {
    username: "admin",
    password: "admin123",
    role: "Admin",
  },
  {
    username: "editor",
    password: "editor123",
    role: "Editor",
  },
  {
    username: "viewer",
    password: "viewer123",
    role: "Viewer",
  },
];

export const authenticateUser = (username, password) => {

  const user = users.find(
    (u) =>
      u.username === username &&
      u.password === password
  );

  if (!user) return null;
```

```

const header = btoa(
  JSON.stringify({
    alg: "HS256",
    typ: "JWT",
  })
);

const payload = btoa(
  JSON.stringify({
    username: user.username,
    role: user.role,
  })
);

const signature = btoa("rbac-secret");

return `${header}.${payload}.${signature}`;
};

```

Save the file.

Step 10

Inside the **utils** folder create a file.

token.js

Paste the following code.

```

export const getUser = () => {
  const token = localStorage.getItem("token");

  if (!token) return null;

  return JSON.parse(
    atob(token.split(".")[1])
  );
};

export const getRole = () => {
  const user = getUser();

  return user?.role;
};

export const isAuthenticated = () => {
  return localStorage.getItem("token") !== null;
};

export const logout = () => {
  localStorage.removeItem("token");
};

```

Save the file.

Step 11

Inside the **routes** folder create a file.

ProtectedRoute.js

Paste the following code.

```
import { Navigate } from "react-router-dom";
import {
  isAuthenticated,
  getRole,
} from "../utils/token";

function ProtectedRoute({
  children,
  allowedRoles,
}) {

  if (!isAuthenticated()) {
    return <Navigate to="/" />;
  }

  const role = getRole();

  if (!allowedRoles.includes(role)) {
    return <Navigate to="/unauthorized" />;
  }

  return children;
}

export default ProtectedRoute;
```

Save the file.

Step 12

Inside the **components** folder create a file.

Navbar.js

Paste the following code.

```
import { logout, getUser } from "../utils/token";
import { useNavigate } from "react-router-dom";

function Navbar() {

  const navigate = useNavigate();

  const user = getUser();

  const handleLogout = () => {
```

```

    logout();

    navigate("/");

};

return (
  <nav>

    <h2>RBAC Authentication</h2>

    <div>

      {user?.username}

      { " | " }

      {user?.role}

      <button onClick={handleLogout}>
        Logout
      </button>

    </div>

  </nav>

);
}

export default Navbar;

```

Save the file.

Step 13

Inside the **pages** folder create a file.

Login.js

Paste the following code.

```

import { useState } from "react";
import { useNavigate } from "react-router-dom";
import { authenticateUser } from "../services/authService";

function Login() {

  const navigate = useNavigate();

  const [username, setUsername] = useState("");
  const [password, setPassword] = useState("");

  const handleLogin = () => {

    const token = authenticateUser(
      username,
      password
    );

    if (!token) {

```

```

        alert("Invalid Credentials");

        return;
    }

    localStorage.setItem("token", token);

    const payload = JSON.parse(
        atob(token.split(".")[1])
    );

    if (payload.role === "Admin")
        navigate("/admin");

    else if (payload.role === "Editor")
        navigate("/editor");

    else
        navigate("/viewer");

};

return (
    <div>

        <h1>RBAC Login</h1>

        <input
            placeholder="Username"
            value={username}
            onChange={ (e) => setUsername(e.target.value) }
        />

        <input
            type="password"
            placeholder="Password"
            value={password}
            onChange={ (e) => setPassword(e.target.value) }
        />

        <button onClick={handleLogin}>
            Login
        </button>

    </div>

);
}

export default Login;

```

Save the file.

Step 14

Inside the **pages** folder create a file.

Admin.js

Paste the following code.

```
import Navbar from "../components/Navbar";

function Admin() {

  return (

    <div>

      <Navbar />

      <h1>Admin Dashboard</h1>

      <h3>User Management</h3>

      <h3>Reports</h3>

      <h3>Settings</h3>

    </div>

  );

}

export default Admin;
```

Save the file.

Step 15

Inside the **pages** folder create a file.

Editor.js

Paste the following code.

```
import Navbar from "../components/Navbar";

function Editor() {

  return (

    <div>

      <Navbar />

      <h1>Editor Dashboard</h1>

      <h3>Edit Posts</h3>

      <h3>Manage Content</h3>

      <h3>Reports</h3>

    </div>

  );

}
```

```
export default Editor;
```

Save the file.

Step 16

Inside the **pages** folder create a file.

Viewer.js

Paste the following code.

```
import Navbar from "../components/Navbar";

function Viewer() {

  return (

    <div>

      <Navbar />

      <h1>Viewer Dashboard</h1>

      <h3>View Posts</h3>

      <h3>View Reports</h3>

      <h3>Profile</h3>

    </div>

  );

}

export default Viewer;
```

Save the file.

Step 17

Inside the **pages** folder create a file.

Unauthorized.js

Paste the following code.

```
import { useNavigate } from "react-router-dom";

function Unauthorized() {

  const navigate = useNavigate();
```

```

return (

  <div>

    <h1>403</h1>

    <h2>Access Denied</h2>

    <button
      onClick={()=>navigate("/") }
    >
      Back to Login
    </button>

  </div>

);

}

export default Unauthorized;

```

Save the file.

Step 18

Open

```

src
↓
App.js

```

Replace the existing code with:

```

import {
  BrowserRouter,
  Routes,
  Route,
} from "react-router-dom";

import Login from "../pages/Login";
import Admin from "../pages/Admin";
import Editor from "../pages/Editor";
import Viewer from "../pages/Viewer";
import Unauthorized from "../pages/Unauthorized";

import ProtectedRoute from "../routes/ProtectedRoute";

import "../App.css";

function App() {

  return (

    <BrowserRouter>

      <Routes>

        <Route
          path="/"
          element={<Login />}

```

```

    />

    <Route
      path="/admin"
      element={
        <ProtectedRoute
          allowedRoles={ ["Admin"]}
        >
          <Admin />
        </ProtectedRoute>
      }
    />

    <Route
      path="/editor"
      element={
        <ProtectedRoute
          allowedRoles={ ["Editor"]}
        >
          <Editor />
        </ProtectedRoute>
      }
    />

    <Route
      path="/viewer"
      element={
        <ProtectedRoute
          allowedRoles={ ["Viewer"]}
        >
          <Viewer />
        </ProtectedRoute>
      }
    />

    <Route
      path="/unauthorized"
      element={<Unauthorized />}
    />

  </Routes>

</BrowserRouter>

);
}

export default App;

```

Save the file.

Step 19

Open

```

src
↓
App.css

```

Replace the existing code with:

```
body{
margin:0;
font-family:Arial,sans-serif;
background:#eef2ff;
}

div{
text-align:center;
padding:20px;
}

input{
width:250px;
padding:10px;
margin:10px;
}

button{
padding:10px 20px;
margin:5px;
cursor:pointer;
background:#4f46e5;
color:white;
border:none;
border-radius:5px;
}

nav{
display:flex;
justify-content:space-between;
align-items:center;
background:#4f46e5;
color:white;
padding:15px;
}

h1{
color:#4f46e5;
}
```

Save the file.

Step 20

Run the application.

```
npm start
```

Step 21

Test the application.

Login using the following credentials:

Admin

Username : admin

Password : admin123
Editor

Username : editor

Password : editor123
Viewer

Username : viewer

Password : viewer123

Verify:

- Admin opens **Admin Dashboard**
 - Editor opens **Editor Dashboard**
 - Viewer opens **Viewer Dashboard**
 - Enter `/admin` in the browser while logged in as Viewer to verify the **403 Access Denied** page.
-

Final Project Structure

rbac-authentication

```
src
├── components
│   └── Navbar.js
├── pages
│   ├── Login.js
│   ├── Admin.js
│   ├── Editor.js
│   ├── Viewer.js
│   └── Unauthorized.js
├── routes
│   └── ProtectedRoute.js
├── services
│   └── authService.js
├── utils
│   └── token.js
├── App.js
├── App.css
├── index.js
└── index.css
```

Experiment 4.1 – Interactive Calendar Scheduling

Step 1

Open Visual Studio Code.

Step 2

Open Terminal.

```
Terminal  
↓  
New Terminal
```

Step 3

Create a new React project.

```
npx create-react-app post-calendar-scheduler
```

If prompted, type:

```
y
```

Step 4

Move inside the project.

```
cd post-calendar-scheduler
```

Step 5

Open the project.

```
code .
```

Step 6

Install FullCalendar packages.

```
npm install @fullcalendar/react @fullcalendar/daygrid @fullcalendar/interaction
```

Step 7

Start the React application.

```
npm start
```

Step 8

Inside the **src** folder create the following folders.

```
src
```

```
components
```

```
data
```

Step 9

Inside the **data** folder create a file.

```
events.js
```

Paste the following code.

```
const events = [  
  {  
    id: "1",  
    title: "Instagram Post",  
    date: "2026-08-05",  
  },  
  {  
    id: "2",  
    title: "LinkedIn Article",  
    date: "2026-08-08",  
  },  
  {  
    id: "3",  
    title: "Facebook Campaign",  
    date: "2026-08-12",  
  },  
  {  
    id: "4",  
    title: "Twitter Post",  
    date: "2026-08-15",  
  },  
];  
  
export default events;
```

Save the file.

Step 10

Inside the **components** folder create a file.

CalendarView.js

Paste the following code.

```
import FullCalendar from "@fullcalendar/react";
import dayGridPlugin from "@fullcalendar/daygrid";
import interactionPlugin from "@fullcalendar/interaction";

import events from "../data/events";

function CalendarView() {

  const handleEventClick = (info) => {

    alert(
      "Selected Event : " +
      info.event.title
    );

  };

  const handleEventDrop = (info) => {

    alert(
      info.event.title +
      " moved to " +
      info.event.start.toDateString()
    );

  };

  return (

    <FullCalendar

      plugins={[
        dayGridPlugin,
        interactionPlugin,
      ]}

      initialView="dayGridMonth"

      editable={true}

      selectable={true}

      events={events}

      eventClick={handleEventClick}

      eventDrop={handleEventDrop}

    />

  );

}

export default CalendarView;
```

Save the file.

Step 11

Open

```
src
↓
App.js
```

Replace the existing code with:

```
import "../App.css";

import CalendarView from "../components/CalendarView";

function App() {
  return (
    <div className="App">
      <h1>
        Interactive Calendar Scheduler
      </h1>
      <CalendarView />
    </div>
  );
}

export default App;
```

Save the file.

Step 12

Open

```
src
↓
App.css
```

Replace the existing code with:

```
body{
margin:0;
font-family:Arial,sans-serif;
background:#eef2ff;
}
```

```
.App{
padding:20px;
text-align:center;
}

h1{
color:#4f46e5;
margin-bottom:20px;
}

.fc{
background:white;
padding:20px;
border-radius:10px;
box-shadow:0 2px 10px rgba(0,0,0,.2);
}
```

Save the file.

Step 13

Run the application.

```
npm start
```

Step 14

Test the application.

- Verify the monthly calendar is displayed.
 - Verify all scheduled posts appear on their respective dates.
 - Click on any event to display its title.
 - Drag an event to another date.
 - Verify the updated schedule.
-

Final Project Structure

post-calendar-scheduler

```
src
├── components
│   └── CalendarView.js
├── data
│   └── events.js
├── App.js
├── App.css
├── index.js
└── index.css
```


Experiment 4.2 – Performance Optimization & Testing

Step 1

Open Visual Studio Code.

Step 2

Open Terminal.

```
Terminal  
↓  
New Terminal
```

Step 3

Create a new React project.

```
npx create-react-app calendar-performance
```

If prompted, type:

```
y
```

Step 4

Move inside the project.

```
cd calendar-performance
```

Step 5

Open the project.

```
code .
```

Step 6

Install the required packages.

```
npm install @testing-library/react @testing-library/jest-dom
```

Step 7

Start the React application.

```
npm start
```

Step 8

Inside the **src** folder create the following folders.

```
src
```

```
components
```

```
tests
```

Step 9

Inside the **components** folder create a file.

```
Calendar.js
```

Paste the following code.

```
import React, { memo } from "react";

function Calendar({ events, onEventClick }) {
  return (
    <div>
      <h2>Post Calendar</h2>

      {events.map((event) => (
        <div
          key={event.id}
          onClick={() => onEventClick(event)}
        >
          {event.title}
        </div>
      ))}
    </div>
  );
}

export default memo(Calendar);
```

Save the file.

Step 10

Open

```
src
```

↓
App.js

Replace the existing code with:

```
import { useState, useMemo, useCallback } from "react";
import Calendar from "../components/Calendar";

import "../App.css";

function App() {

  const [search, setSearch] = useState("");

  const events = [
    { id: 1, title: "React Workshop" },
    { id: 2, title: "Redux Seminar" },
    { id: 3, title: "Machine Learning" },
    { id: 4, title: "JavaScript Basics" },
  ];

  const filteredEvents = useMemo(() => {

    return events.filter((event) =>
      event.title
        .toLowerCase()
        .includes(search.toLowerCase())
    );

  }, [search]);

  const handleClick = useCallback((event) => {

    alert(event.title);

  }, []);

  return (

    <div className="App">

      <h1>Calendar Performance Optimization</h1>

      <input
        type="text"
        placeholder="Search Event"
        value={search}
        onChange={(e) =>
          setSearch(e.target.value)
        }
      />

      <Calendar
        events={filteredEvents}
        onEventClick={handleClick}
      />

    </div>

  );

}
```

export default App;

Save the file.

Step 11

Open

```
src
↓
App.css
```

Replace the existing code with:

```
body{
margin:0;
font-family:Arial,sans-serif;
background:#eef2ff;
}

.App{
text-align:center;
padding:30px;
}

input{
width:300px;
padding:10px;
margin-bottom:20px;
}

h1{
color:#4f46e5;
}

div{
margin:10px;
}
```

Save the file.

Step 12

Inside the **tests** folder create a file.

```
Calendar.test.js
```

Paste the following code.

```
import { render, screen } from "@testing-library/react";
import Calendar from "../components/Calendar";

test("renders calendar heading", () => {
  render(
    <Calendar
      events={}
      onEventClick={() => {}}
    />
  )
})
```

```
    />  
  );  
  expect (  
    screen.getByText ("Post Calendar")  
  ).toBeInTheDocument ();  
});
```

Save the file.

Step 13

Run the application.

```
npm start
```

Step 14

Run the test cases.

```
npm test
```

Press

a

to run all test cases.

Step 15

Verify the application.

- Calendar heading is displayed.
 - Events are listed.
 - Search filters events.
 - Clicking an event displays an alert.
 - Test cases execute successfully.
 - Application renders without unnecessary re-renders.
-

Final Project Structure

```
calendar-performance
```

src

